Machine Learning and Inductive Inference - Lecture 02

Lecture overview

Lecture 02 introduces supervised learning as a paradigm where data and desired outputs are used to derive a program (hypothesis), contrasting with traditional programming that uses data and a program to generate output. The lecture outlines a five-step machine learning process: data collection, preprocessing, feature construction, model building, and evaluation. A critical component is feature construction, where properties like BPM or energy are transformed into fixed-length vectors; the choice of features significantly impacts performance as there is no ground truth for selection. Data must be split into training, validation, and testing sets to ensure the model generalizes to unseen examples rather than overfitting by merely memorizing the training data. The lecture distinguishes between eager learners, which build a global model during training, and lazy learners like k-Nearest Neighbors (kNN), which defer computation until prediction time by comparing new inputs directly to stored training instances.

The core of the lecture focuses on the mechanics of the kNN algorithm, an instance-based method that classifies a query point by identifying its k closest neighbors in feature space using specific distance metrics such as Euclidean, Manhattan, Hamming, or Jaccard distance. A key exam concept is the necessity of normalizing features to prevent variables with larger numerical ranges from skewing distance calculations. The algorithm aggregates neighbor labels via majority voting or distance-weighted schemes, where closer neighbors exert more influence. The value of k acts as a control for model complexity: small k values create rough decision surfaces prone to overfitting, while large k values produce smoother surfaces that risk underfitting. Geometrically, kNN partitions the feature space into Voronoi regions, and its effectiveness relies heavily on appropriate distance measures and handling the curse of dimensionality, despite advantages like fast training times and flexibility in output types.

Slide 1: Lecture 2: Supervised

Lecture 2 introduces supervised learning and the k-Nearest Neighbors (kNN) algorithm within the context of machine learning and inductive inference. The session is presented by Sieben Bocklandt, utilizing slide decks originally developed by Jesse Davis for the Principles of Machine Learning course.

Slide 2: - Understand the basic goal of inductive supervised learning

This slide outlines the primary learning objectives for the lecture, starting with understanding the basic goal of inductive supervised learning. It further lists essential topics including feature construction, the definition of a hypothesis, and the components of a learning algorithm. The goals also cover the importance of the kNN algorithm, its mechanics regarding distance measures and decision surfaces, and strategies to address its associated pitfalls.

Slide 3: Supervised learning

Supervised learning starts with labeled examples: each input has a target output. A learner uses those pairs to fit a model that predicts targets for new inputs.

Slide 4: Faculteit, departement, dienst …

This slide places supervised learning within the broader machine-learning hierarchy. The defining property here is that the training data supplies the desired labels or values, which the model learns to predict.

Slide 5: Faculteit, departement, dienst …

The classification example treats the target as a class label. Positive and negative examples show how labeled cases constrain a decision rule; the rule is then applied to an unseen case whose label is unknown.

Slide 6: Faculteit, departement, dienst …

Slide 6 introduces an example classification problem involving positive and negative examples. The specific task asks how to classify a "solid red circle in a polygon." No further definitions, rules, or professor explanations are provided in the source material for this slide.

Slide 7: Faculteit, departement, dienst …

Slide 7 presents an example classification problem distinguishing between positive and negative cases. The positive example is defined as a solid red circle located within any polygon, while the negative example specifies a solid red circle situated inside a regular polygon. The slide poses the question of how to classify these specific geometric configurations based on the provided criteria.

Slide 8: Faculteit, departement, dienst …

Slide 8 presents an example classification problem involving geometric shapes. It contrasts positive examples, such as a solid red circle inside any polygon, with negative examples where the circle is restricted to a regular polygon. The slide poses the specific question of how to classify these instances based on the distinction between general and regular polygons.

Slide 9: Traditional programming

Slide 9 contrasts traditional programming with supervised machine learning by illustrating their distinct input-output flows. In traditional programming, a computer combines data and a program to produce an output. Conversely, in supervised learning, the computer uses data and the desired output to generate the program itself.

Slide 10: - Given a set of data consisting of < x , y > with

Supervised learning operates on a dataset composed of pairs denoted as <x, y>. In this framework, x represents a set of features, while y is the label associated with those specific features. This pairing of inputs and their corresponding outputs forms the fundamental data structure for supervised learning tasks.

Slide 11: - Given a set of data consisting of < x , y > with

Supervised learning involves training a model using a dataset of input features (x) and their corresponding labels (y). Unlike traditional programming, which combines data with a fixed program to produce output, machine learning uses paired inputs and outputs to learn the underlying program or hypothesis. The goal is to derive a function that can accurately map new feature sets (x') to predicted outputs (y').

Slide 12: How to do supervised

Slide 12 is titled "How to do supervised learning" and includes the footer text "Faculteit, departement, dienst … 12." No specific content, definitions, or procedural steps regarding supervised learning are provided in the slide text or the aligned professor transcript for this section.

Slide 13: - Music playlist consisting of songs used as background music

Slide 13 introduces a running example centered on a music playlist composed of songs utilized as background music. This specific scenario serves as the foundational context for subsequent discussions within the lecture series. No additional details or professor explanations were provided for this particular slide in the available source material.

Slide 14: - Music playlist consisting of songs used as background music

The music-playlist example makes the labels concrete: tracks a listener accepts act as positive examples and rejected tracks as negative examples. A model can use those examples to predict which unseen tracks may fit the listener's preferences; the labels reflect observed choices, not an objective quality of the music.

Slide 15: - Music playlist consisting of songs used as background music

This slide introduces a running example involving a music playlist of background songs where the goal is to automatically add new tracks. It poses the central problem-solving question: what steps are required to achieve this automatic addition? The content serves as a practical context for applying subsequent concepts without providing a specific solution in this section.

Slide 16: - Music playlist consisting of songs used as background music

Slide 16 introduces a running example of automatically building a background music playlist by filtering songs based on features like BPM and energy levels. The process requires five steps: data collection, preprocessing, feature construction, model building, and evaluation. A hypothesis space divides the feature space into positive and negative regions to classify new songs, such as determining if a track with 125 BPM fits the desired low-energy criteria.

Slide 17: - Music playlist consisting of songs used as background music

This slide introduces a running example of automatically adding new songs to a background music playlist. To solve this problem, the lecture outlines a five-step process: data collection, data preprocessing, feature construction, model building, and model evaluation. This framework serves as a practical illustration for understanding the focus of the class and academic research compared to industry practice.

Slide 18: - What to collect? (Images, Sensor readings, Documents, …) → Structured?

The first data collection question asks what specific data to gather, such as images or sensor readings, and how structured it should be. The professor illustrates this by contrasting general image collection with targeted requests like "images of cute dogs," emphasizing that defining the scope determines the required level of structure. This decision initiates a broader framework addressing where, how, how much, and where to store data while considering legal and practical constraints.

Slide 19: - Evaluating, filtering, manipulating and encoding data to make sure the ML model

Data preprocessing involves evaluating, filtering, manipulating, and encoding data to ensure machine learning models can effectively utilize it. Key techniques include handling missing values (e.g., removing examples or adding mean values), scaling variables to uniform forms, removing outliers like random noise, and converting non-numerical data into numerical formats. Additionally, the most critical step is dividing the dataset into three distinct sets: training for model development, validation for tuning, and testing for final evaluation.

Slide 20: - Which features can describe a song?

Slide 20 introduces the topic of feature construction by posing the question of which features can describe a song. As no specific features or definitions are provided in the slide text or aligned transcript, this section serves solely as a heading to initiate the discussion on identifying musical characteristics for analysis.

Slide 21: - Which features can describe a song?

Feature construction involves defining attributes that describe a song, such as genre, duration, tempo (BPM), key, loudness (dB), acousticness, energy, and whether it is instrumental. These features are categorized into objective values, like duration and tempo, and subjective values, including acousticness and energy. Defining these features is essential because they characterize the problem and form the basis for building and evaluating models.

Slide 22: - Features describe the problem

Features describe the problem and are known by various names depending on the domain: attributes in databases, covariates in statistical learning, variables in probabilistic learning, and KPIs in business applications. Defining features is a critical skill rather than a mechanical process because there is no ground truth; poor feature selection directly results in poor model performance. Additionally, features can take different forms, such as boolean values represented as true/false or 1/0 in code.

Slide 23: - Boolean (1/0 ; True/False)

Slide 23 outlines five feature types: Boolean (e.g., Instrumental={True, False}), Nominal with no value relationship (e.g., Genre), Linear/Ordered with structured values (e.g., Energy={Low, Medium, High}), Numerical integers (e.g., BPM), and Hierarchical features with partial IS-A ordering. The professor clarifies that nominal categories like music genres lack inherent order, whereas ordered features like energy levels possess a defined structure from low to high.

Slide 24: Features: properties that describe the problem

Features are properties that describe a problem for any given data point, such as a song or image. These features form a fixed-length vector, ensuring every data point shares the same structure with a consistent number of attributes like genre or beat. While missing data may occur and require specific handling strategies, this fixed structure allows models to reliably process new inputs using a defined set of features.

Slide 25: Features: properties that describe the problem

Features are properties that describe a problem by converting a new data point, such as a song or image, into a fixed-length feature vector. This process involves defining m specific attributes to represent the data; for example, a song can be described using features like title, genre, BPM, key, and whether it is instrumental.

Slide 26: Features: properties that describe the problem

Features are properties that describe a problem, used to construct a fixed-length feature vector for any data point like a song. This involves defining specific attributes such as title, genre, BPM, key, and instrumental status, where each feature has a distinct domain type including string, nominal, numerical, or boolean. In supervised learning contexts, these features are paired with a label, such as whether the item belongs to a playlist, to fully characterize the data instance.

Slide 27: Features: properties that describe the problem

Features are properties that describe a problem, where each new data point is defined by m features with specific value domains based on their type. Consequently, every example is represented as a fixed-length vector of these m feature values, potentially accompanied by a class label. For instance, a song can be characterized by string titles, nominal genres, numerical BPMs, and boolean attributes like instrumental status.

Slide 28: - Each axis is a feature with its own domain. This example is simplified (only 2 features)

In a feature space, each axis represents a distinct feature with its own specific domain. The provided example simplifies this concept by visualizing only two features: BPM and Energy. This two-dimensional representation illustrates how data points are plotted based on their values along these independent axes.

Slide 29: - Each axis is a feature with its own domain. This example is simplified (only 2 features)

In a feature space, each axis represents a distinct feature with its own domain. This example simplifies the concept to two features: BPM and Energy. A new song with specific values (BPM=125, Energy=2.7) is located within this two-dimensional space based on its coordinates along these axes.

Slide 30: Representing models

Slide 30 introduces the topic of representing models within an organizational context. The slide explicitly lists "Faculteit" (Faculty), "departement" (Department), and "dienst" (Service) as key entities involved in this representation. No further explanation, definitions, or examples are provided in the aligned transcript for this specific section.

Slide 31: Faculteit, departement, dienst …

The plot shows labeled music examples in a two-feature space with BPM on one axis and Energy on the other. Candidate hypotheses divide that space into regions, illustrating that several decision boundaries can fit the same training labels.

Slide 32: Faculteit, departement, dienst …

This build applies one candidate hypothesis to the BPM-Energy plane and labels regions as positive or negative. A hypothesis is one possible classifier; it is selected from the larger hypothesis space.

Slide 33: Faculteit, departement, dienst …

The question marks are unlabeled songs. Each candidate boundary assigns them a class, demonstrating how different hypotheses consistent with training examples can make different predictions.

Slide 34: Set of legal hypotheses: in this example only straight lines

Slide 34 defines the set of legal hypotheses for this specific example as restricted to straight lines only. The slide visualizes the hypothesis space using a graph with BPM and Energy axes, displaying data points marked as positive (+), negative (-), or unknown (?). It specifically illustrates "Hypothesis 1" within this constrained space of linear functions.

Slide 35: Set of legal hypotheses: in this example only straight lines

Slide 35 defines the set of legal hypotheses for this specific example as being restricted to only straight lines. The slide visualizes the hypothesis space using a graph with BPM and Energy axes, displaying data points marked with positive (+), negative (-), and unknown (?) labels. It illustrates two distinct candidate models, labeled Hypothesis 1 and Hypothesis 2, which represent potential straight-line fits within this constrained space.

Slide 36: Set of legal hypotheses: in this example only straight lines

Slide 36 defines the set of legal hypotheses for this specific example as being restricted to straight lines. The slide visualizes a hypothesis space containing three distinct linear candidates, labeled Hypothesis 1, Hypothesis 2, and Hypothesis 3. These hypotheses are plotted against axes representing BPM and Energy, illustrating how different straight-line models fit the provided data points.

Slide 37: - Assume a fixed form for the relationship between the features and the target

This slide defines an approach where a fixed functional form for the relationship between features and the target is assumed before observing any data. This constraint results in hypothesis classes of a fixed size, limiting the model's flexibility to specific predefined structures rather than adapting freely to the dataset.

Slide 38: - Assume a fixed form for the relationship between the features and the target

This slide defines fixed-size hypothesis classes where the relationship between features and targets assumes a specific structural form prior to observing data. Within this framework, the model's structure remains constant while only its parameters vary, such as using straight lines or hyperplanes to divide feature space. Canonical examples of this approach include linear regression, logistic regression, and Naïve Bayes.

Slide 39: - Structure and form of the model changes during learning, often becoming more

Variable-size hypothesis classes differ from fixed-structure models because their form changes during learning, often becoming more complex as data increases. Canonical examples include decision trees, which grow by adding new nodes, and rule sets, which expand by incorporating additional rules. Unlike models with a constant mathematical description where only weights vary, these structures dynamically adapt their complexity to the learning process.

Slide 40: Learning a model

This page marks the transition to learning a model. The model is the general rule inferred from the training examples and is later used to predict outputs for new cases.

Slide 41: - Given

The overall learning objective for binary classification assumes a given set of positive and negative examples. These examples are drawn independently at random from an unknown probability distribution. This setup establishes the fundamental data conditions required to train a binary classifier.

Slide 42: - Given

Given positive and negative examples drawn independently from an unknown probability distribution, the objective is to learn a model that classifies most positives correctly while excluding negatives. Crucially, the model must generalize to correctly label future examples rather than merely memorizing the training set, which would constitute overfitting. This binary classification framework can be extended to scenarios with more than two classes.

Slide 43: - Suppose I have a problem with 4 binary features and 2 classes

This slide illustrates the fundamental challenge of learning by analyzing a problem with four binary features and two classes, which creates a feature space of 16 possible examples. Despite this small space, there are 65,536 distinct ways to label these examples, meaning observing only a subset of data leaves many potential models consistent with the training set. Consequently, deciding which model is best remains difficult because partial observation does not sufficiently rule out alternative hypotheses.

Slide 44: - We need to make assumptions: experience alone doesn’t allow us to make

Experience alone cannot determine conclusions for unseen data instances, necessitating assumptions known as inductive bias to guide model selection within a vast hypothesis space. Inductive bias consists of two types: restriction, which limits the hypothesis space to specific forms like rules or linear models, and preference, which imposes an ordering on hypotheses, such as favoring simpler models over complex ones.

Slide 45: - At high-level, algorithms learn one of these:

At a high level, algorithms learn one of three primary structures: a function that predicts a class or real value, such as decision trees and linear regression; a conditional probability distribution P(Y|X), exemplified by logistic regression and neural networks; or a joint probability distribution P(X,Y), as seen in Naïve Bayes. These categories define the fundamental goals of learning algorithms.

Slide 46: - At high-level, algorithms learn one of these:

At a high level, algorithms learn one of three targets: a function predicting class or real values (e.g., decision trees), a conditional probability distribution P(Y|X) (e.g., logistic regression), or a joint probability distribution P(X,Y) (e.g., Naïve Bayes). Learning a model follows a two-step approach involving searching through the hypothesis space and evaluating candidate hypotheses. This process continues by tweaking parameters until a satisfactory model is found, balancing simplicity against performance costs.

Slide 47: - Discrete:

Discrete model building involves selecting a specific architecture or configuration from a hypothesis space using combinatorial search methods like best-first, depth-first, or beam-search. This process addresses decisions regarding hyperparameter choices and includes techniques such as decision trees, feature selection, and rule learning to navigate the available options effectively.

Slide 48: - Discrete:

Searching the hypothesis space involves two approaches: discrete and continuous. Discrete search determines "what model to build" using combinatorial methods like best-first or beam-search for choices such as architecture, hyperparameters, decision trees, feature selection, and rule learning. In contrast, continuous search addresses "what numbers the model should contain," utilizing optimization techniques like gradient descent to tune weights and parameters in fixed-structure models such as neural networks.

Slide 49: - Typically we have two high-level components:

Evaluating candidate models typically involves assessing two high-level components. The first component measures how well the model fits the data, while the second evaluates the complexity of the model itself. These factors are central to determining the overall quality and suitability of a candidate model.

Slide 50: - Typically we have two high-level components:

Evaluating candidate models involves balancing two high-level components: how well the model fits the data and its complexity. The goal is to trade off good fit against bad complexity, ideally achieving a perfect fit with minimal complexity. In rule learning, this is quantified by calculating positives covered minus negatives covered minus the number of literals; for instance, a simpler rule covering 20 positives and 10 negatives may score higher than a more complex rule covering 21 positives and 9 negatives.

Slide 51: - A loss function quantifies the impact or consequence of an error (incorrect prediction)

A loss function quantifies the impact or consequence of an incorrect prediction, such as 0/1-loss which assigns 0 for correct and 1 for incorrect predictions; its average represents the classification error rate. The primary goal is to find a hypothesis h within the hypothesis space $\mathcal{H}$ that minimizes this loss on the data while simultaneously minimizing model size or complexity. This optimization balances reducing prediction error with controlling the weight of the model's size.

Slide 52: - Lazy learners: there is no explicit model

Lazy learners do not construct an explicit model during training; instead, they build a new model on demand for each specific query by comparing the surroundings of the input to neighboring examples. This approach pushes all computational work to prediction time rather than learning phase. Practical applications include recommender systems that suggest similar items based on current queries and medical case comparisons, such as analyzing breast cancer cases against historical data.

Slide 53: - Lazy learners: there is no explicit model

Lazy learners do not create an explicit model during training; instead, they build a new model on demand for each query, pushing all computational work to prediction time. In contrast, eager learners construct a single global model that applies to all examples, completing the necessary work at training time. This distinction highlights the fundamental difference in when and how these two approaches perform their modeling tasks.

Slide 54: A real example of learning a model:

Slide 54 introduces the k-nearest neighbors (kNN) algorithm as a concrete example of learning a model. The slide explicitly identifies kNN to illustrate how a machine learning model can be constructed and applied in practice. No further technical details, definitions, or examples regarding the kNN mechanism are provided in the source material for this specific section.

Slide 55: Faculteit, departement, dienst …

Slide 55 presents the title "Faculteit, departement, dienst …" and references an example by Josh Tenenbaum. No additional explanatory content, definitions, or professor commentary is provided in the aligned source material for this specific slide. Consequently, the slide serves solely as a reference point to this named example without further elaboration on organizational structures or related concepts.

Slide 56: Tufa

Slide 56 presents an example of tufa attributed to Josh Tenenbaum and prompts viewers to identify additional instances of this feature. The slide includes administrative text referencing a faculty, department, or service but provides no further definitions, descriptions, or technical details regarding tufa itself.

Slide 57: Faculteit, departement, dienst …

Slide 57 presents the title "Faculteit, departement, dienst …" and references an example by Josh Tenenbaum. No further explanatory content, definitions, or professor commentary is provided in the aligned source material for this specific slide. Consequently, the slide serves solely as a heading introducing this example without additional context or details available in the current text.

Slide 58: - If it walks like a duck and quacks like a duck, then it’s probably a duck

Slide 58 introduces instance-based learning intuitively using the analogy that if something behaves like a duck, it is likely a duck. The slide illustrates this concept with a scatter plot of BPM and Energy data points labeled as positive (+) or negative (-), where an unknown query point (?) surrounded by positive instances is classified as "Probably a positive." This demonstrates how classification relies on the characteristics of nearby stored examples.

Slide 59: - If it walks like a duck and quacks like a duck, then it’s probably a duck

This slide introduces instance-based learning through the analogy that if an entity behaves like a duck, it is likely a duck. The accompanying diagram illustrates this concept by plotting data points labeled as positive (+) or negative (-) against BPM and Energy axes. New instances marked with question marks are classified as "probably a positive" or "probably a negative" based on their proximity to these existing labeled examples.

Slide 60: - If it walks like a duck and quacks like a duck, then it’s probably a duck

Slide 60 introduces instance-based learning through the analogy that if an entity behaves like a duck, it is likely a duck. The slide visually represents this concept using a dataset of positive and negative instances plotted against BPM and Energy axes to classify unknown cases as probably positive or negative based on their proximity to known examples.

Slide 61: - Recommender systems

Slide 61 highlights the practical importance of nearest neighbor methods by listing diverse applications such as recommender systems, medical case comparison, web and image search, news articles, plagiarism detection, weather forecasting, and Retrieval Augmented Generation (RAG). The slide emphasizes that despite appearing trivial, understanding these techniques is essential due to their widespread utility across these critical domains.

Slide 62: Let T={(xi, yi)} where i ∈ {1,..,n}

This slide introduces a first classification attempt based on memorizing the training dataset $T = \{(x_i, y_i)\}$. To classify a new example e, the method checks if an identical instance exists in T by comparing all d attributes; if a match is found, it returns the corresponding label y_i. If no exact match exists, the algorithm returns an unknown result (?).

Slide 63: Let T={(xi, yi)} where i ∈ {1,..,n}

This slide introduces a first classification attempt that memorizes the training set $T=\{(x_i, y_i)\}$ by returning the label y_i only if an exact match is found for all features of example e. If no such instance exists in T, the method returns an unknown result (?). This approach is fundamentally limited because it cannot generalize beyond the specific training data and relies on a very restrictive exact-match condition.

Slide 64: - Find the k nearest labelled examples and return their combined decision

To classify an example e using k-Nearest Neighbors, the algorithm identifies the k nearest labeled examples from the training set T. It maintains a priority queue of size k, updating it by adding any example (x_j, y_j) whose distance to e is less than the current maximum distance in the queue. The final prediction is generated by combining the labels (y_j) of the k examples stored in the priority queue.

Slide 65: Faculteit, departement, dienst …

Slide 65 presents an example of a 1-Nearest Neighbor (1-NN) classification applied to BPM Energy data. The visual content displays a sequence of positive (+) and negative (-) class labels plotted against numerical values ranging from 0 to 200, illustrating the distribution of these categories within the dataset. No additional explanatory details or definitions are provided in the aligned transcript for this specific slide.

Slide 66: Faculteit, departement, dienst …

Slide 66 presents an example of the 1-Nearest Neighbor (1-NN) algorithm applied to BPM Energy data, visualized across a scale from 0 to 200. The diagram displays four distinct rows containing sequences of positive (+), negative (-), and unknown (?) symbols, alongside a labeled queue ranging from 'a' to 'z'. No specific definitions or procedural explanations for this example are provided in the accompanying transcript.

Slide 67: Faculteit, departement, dienst …

Slide 67 presents an example of the 1-Nearest Neighbor (1-NN) algorithm applied to BPM Energy data, visualized with a queue labeled 'a' through 'z'. The slide displays a sequence of positive and negative values alongside a distance metric indicating "Dist a 10 ?". No further explanation or definitions are provided in the aligned transcript for this specific fragment.

Slide 68: Faculteit, departement, dienst …

Slide 68 presents an example of the 1-Nearest Neighbor (1-NN) algorithm applied to BPM Energy data, visualized with a queue labeled 'a' through 'z'. The slide displays a sequence of positive and negative values alongside energy measurements ranging from 0 to 200. It specifically highlights a calculation for "Dist b" resulting in the value 8, illustrating how distance is determined within this dataset context.

Slide 69: Faculteit, departement, dienst …

Slide 69 presents an example of the 1-Nearest Neighbor (1-NN) algorithm applied to a dataset involving BPM and Energy metrics. The visual content displays a queue labeled with letters 'a' through 'z' alongside binary classification symbols (+/-), illustrating a specific instance where the distance for item 'c' is calculated as 3. No additional professor explanation or definitions are provided in the aligned source material for this slide.

Slide 70: Faculteit, departement, dienst …

Slide 70 presents an example of the 1-Nearest Neighbor (1-NN) algorithm applied to BPM Energy data, visualized with a queue labeled 'a' through 'z'. The diagram displays a sequence of positive and negative values alongside a numerical scale ranging from 0 to 200. It specifically highlights a query point 'e' with a distance value of 2, indicated by a question mark, illustrating the classification or retrieval process within this dataset.

Slide 71: Faculteit, departement, dienst …

Slide 71 presents an example of the 1-Nearest Neighbor (1-NN) algorithm applied to a dataset involving BPM and Energy metrics. The visual data displays a sequence of positive and negative classifications alongside a queue labeled with letters from 'a' to 'z'. A specific query point, marked as 'e', is highlighted with a distance value of 2, illustrating the calculation process for determining the nearest neighbor in this classification context.

Slide 72: Faculteit, departement, dienst …

Slide 72 illustrates a 1-Nearest Neighbor (1-NN) classification example using BPM Energy data points labeled as positive (+) or negative (-). The diagram identifies 'e' as the nearest neighbor to the query point '?', with a distance of 2. Consequently, the algorithm assigns the label of 'e', which is negative (-), to the unknown instance.

Slide 73: Faculteit, departement, dienst …

Slide 73 presents an example of the 3-Nearest Neighbors (3-NN) algorithm applied to Business Process Management (BPM) energy data. The visual displays a sequence of positive (+) and negative (-) values across four rows, culminating in a query point marked with a question mark (?). A queue labeled with letters from 'a' through 'z' is listed below the data matrix, likely representing the dataset elements used for the classification or prediction task.

Slide 74: Faculteit, departement, dienst …

Slide 74 presents an example of the 3-Nearest Neighbors (3-NN) algorithm applied to Business Process Mining (BPM). The slide displays a dataset containing energy values, binary classifications (+/-), and a queue of items labeled 'a' through 'z'. It specifically highlights a distance calculation for item 'a', noting a value of 10 with an unknown result indicated by a question mark.

Slide 75: Faculteit, departement, dienst …

Slide 75 presents an example of the 3-Nearest Neighbors (3-NN) algorithm applied to BPM Energy data, visualized with positive and negative class markers across a numerical range. The slide includes a queue labeled with letters 'a' through 'z' and lists specific distance values for items 'b' (8) and 'a' (10), ending with an unknown query point marked by a question mark. No additional professor explanation or definitions are provided in the aligned source material.

Slide 76: Faculteit, departement, dienst …

Slide 76 presents an example of the 3-Nearest Neighbors (3-NN) algorithm applied to BPM Energy data, visualized with positive and negative class markers across a queue labeled 'a' through 'z'. The slide illustrates distance calculations for specific instances, showing distances of 3, 8, and 10 for items c, b, and a respectively, relative to an unknown query point. No additional professor explanation or definitions are provided in the aligned source material.

Slide 77: Faculteit, departement, dienst …

Slide 77 presents an example of the 3-Nearest Neighbors (3-NN) algorithm applied to a dataset involving BPM, Energy, and Queue metrics. The slide displays data points labeled with positive (+) and negative (-) classes alongside a queue sequence from 'a' to 'z'. It specifically illustrates distance calculations for a query point, listing distances of 3 to points 'c' and 'd', and 8 to point 'b', to determine the nearest neighbors.

Slide 78: Faculteit, departement, dienst …

Slide 78 presents an example of the 3-Nearest Neighbors (3-NN) algorithm applied to a BPM Energy dataset. The visual data includes a queue labeled with letters 'a' through 'z' and specific distance calculations for items 'e', 'c', and 'd'. No additional professor explanation or definitions are provided in the aligned transcript for this section.

Slide 79: Faculteit, departement, dienst …

Slide 79 presents an example of the 3-Nearest Neighbors (3-NN) algorithm applied to a BPM Energy dataset. The visual data includes a queue labeled with letters 'a' through 'z' and specific distance calculations for neighbors e, f, and d, which are listed as 2, 3, and 3 respectively. No additional professor explanation or definitions accompany this slide content.

Slide 80: Faculteit, departement, dienst …

Slide 80 illustrates a 3-Nearest Neighbors (3-NN) classification example using BPM Energy data. The process identifies the three closest neighbors to the query point as 'e' (distance 2), 'c' (distance 3), and 'd' (distance 4). By combining the labels of these specific neighbors, the algorithm determines the final classification result is positive (+).

Slide 81: - Find the k nearest labelled examples and return their combined decision

The k-Nearest Neighbors (kNN) algorithm classifies a test example by identifying the k closest training instances using a distance measure and combining their labels to form a prediction. While simple majority voting is common, the professor notes that distance can serve as a weighting parameter where closer neighbors exert more influence. A critical concern highlighted for exams is handling features with different scales; without normalization techniques like min-max normalization, distance calculations become skewed by larger numerical ranges.

Slide 82: What distances are possible?

Slide 82 poses the question "What distances are possible?" and lists the terms "Faculteit," "departement," and "dienst." No further explanation, definitions, or examples regarding these distances or terms are provided in the aligned transcript or slide text.

Slide 83: - Similarities: Higher number = more similar → neighbors are most similar

Similarities and distances operate inversely: higher similarity values indicate greater resemblance, while higher distance values indicate objects are farther apart, making neighbors the closest points. Distances must satisfy four constraints: non-negativity, zero self-distance, symmetry, and the triangle inequality. While distances can be converted to similarities using the formula $sim(i,j) = 1 / (1 + Dist(i,j))$, the reverse conversion is not possible because these mathematical constraints apply only to distances, not similarities.

Slide 84: - All discrete features: Hamming distance counts the number of features

For all discrete features, Hamming distance counts the number of features on which two examples differ by assigning zero if values match and one if they do not. A higher Hamming distance indicates more differing feature values between the instances. The slide also lists Manhattan and Euclidean distances as classic measures for real-valued features, while the professor notes that advanced metrics like value difference exist specifically for probabilities.

Slide 85: - Value difference metric: attribute values are close if they make similar

The value difference metric defines attribute values as "close" when they result in similar predictions. This approach is specifically designed to handle datasets containing mixed continuous and discrete attributes. The slide lists examples such as faculty, department, and service units while categorizing this concept under more advanced measures.

Slide 86: - Jaccard similarity:

Jaccard similarity measures set comparison by calculating the size of the intersection divided by the size of the union. Jaccard distance is defined as one minus the Jaccard similarity, representing how distant two sets are. These formulas appear on the exam formula sheet; for example, if the intersection of sets S1 and S2 has a size of 3, the resulting similarity is 0.3 and the distance is 0.7.

Slide 87: - Edit distance for strings: the number of required edits to transform a string into another

Edit distance for strings is defined as the number of required edits to transform one string into another. For instance, transforming "learning" to "leaning" requires one edit (removing 'r'), while changing "learning" to "carving" requires three edits: removing 'l', replacing 'e' with 'c', and replacing 'n' with 'v'. This concept applies to structured objects like text, distinct from numerical vector similarities such as cosine similarity.

Slide 88: How can we combine

Slide 88 poses the central question of how to combine neighbors' labels within a hierarchical structure. The slide specifically lists "Faculteit," "departement," and "dienst" as examples of these organizational levels. No further explanation or methodology for combining these labels is provided in the aligned transcript or slide text.

Slide 89: - Majority vote

While standard k-NN uses majority voting, this approach is insufficient for cases like 5-NN where neighbor distances vary. The solution is distance-weighted k-NN, which predicts the class by maximizing the sum of weights for each label, where the weight w_i is inversely proportional to the squared distance between the query point and the neighbor.

Slide 90: - Average of the nearest neighbors

Slide 90 introduces the method of using the average of the nearest neighbors to predict real values within the k-Nearest Neighbors (kNN) framework. This approach distinguishes regression tasks, which output continuous numerical results, from classification tasks that typically assign discrete categories. The core mechanism involves calculating the mean of the target values from the identified nearest neighbors to generate the final prediction.

Slide 91: - Average of the nearest neighbors

While k-NN is commonly used for classification, it can also predict real numerical values through regression by calculating the average of the nearest neighbors' values. This approach addresses limitations in standard classification where distance weighting might be necessary to account for varying proximity among neighbors. Instead of assigning a single class label, the algorithm computes the mean of the target values from the selected k-nearest neighbors to generate a continuous prediction.

Slide 92: - Average of the nearest neighbors

To predict real values using k-nearest neighbors (kNN), a local regression model serves as an alternative solution to simply averaging the nearest neighbors. This approach involves learning a specific model on the k nearest neighbors and then utilizing that local model to generate the final prediction.

Slide 93: How expressive is kNN?

Slide 93 poses the question of how expressive the k-Nearest Neighbors (kNN) algorithm is. The slide text contains only this title and footer information, with no accompanying definitions, explanations, or examples provided in the source material. Consequently, no specific details regarding the expressiveness of kNN can be derived from this section alone.

Slide 94: - Expressive = expressiveness of kNN’s hypothesis space

Expressiveness refers to the complexity of concepts that kNN's hypothesis space can represent and how the algorithm partitions the feature space. It addresses the question of how complex the resulting models are. To illustrate this partitioning, the professor describes using perpendicular bisectors between nodes to divide the feature space into decision regions.

Slide 95: Faculteit, departement, dienst …

Slide 95 introduces the terms "Faculteit," "departement," and "dienst" as organizational units. The slide explicitly directs the audience to examine these concepts from a geometric perspective, though no further definitions or specific geometric details are provided in the source material for this section.

Slide 96: Faculteit, departement, dienst …

Slide 96 introduces the terms "Faculteit," "departement," and "dienst" while proposing to examine these concepts geometrically. No further definitions, explanations, or specific details regarding these organizational units are provided in the available source material for this section.

Slide 97: Faculteit, departement, dienst …

Slide 97 introduces the terms "Faculteit," "departement," and "dienst" while proposing to examine these concepts geometrically. No further definitions, explanations, or specific details regarding these organizational units are provided in the available source material for this section.

Slide 98: Faculteit, departement, dienst …

Slide 98 introduces the terms "Faculteit," "departement," and "dienst" as a lead-in to a geometric perspective on these organizational structures. The slide explicitly states, "Let's look at this geometrically," indicating that the subsequent analysis will visualize the relationships between these entities using geometric concepts. No further definitions or specific details regarding these terms are provided in the available source material for this section.

Slide 99: Faculteit, departement, dienst …

Slide 99 introduces the terms "Faculteit," "departement," and "dienst" while proposing to examine these concepts from a geometric perspective. The slide text explicitly states the intention to analyze the relationship between these organizational units geometrically, though no further definitions or specific details are provided in the available source material for this section.

Slide 100: Faculteit, departement, dienst …

Slide 100 introduces the terms "Faculteit," "departement," and "dienst" while proposing a geometric perspective to analyze these concepts. The slide explicitly states, "Let's look at this geometrically," indicating that the subsequent discussion will interpret these organizational structures through a spatial or visual framework. No further definitions or specific details regarding these terms are provided in the current source material.

Slide 101: Faculteit, departement, dienst …

Slide 101 introduces a geometric perspective on the concepts of "Faculteit," "departement," and "dienst" by utilizing a Voronoi diagram. The slide explicitly directs the analysis to visualize these organizational structures through this specific geometric framework, though no further definitions or examples are provided in the source material.

Slide 102: Faculteit, departement, dienst …

A Voronoi diagram partitions space into zones where each point is closest to a specific training node, effectively creating a decision surface for 1-NN classification. A new example is assigned the label of the node within its zone, resulting in a rough boundary when visualized with features like beats per minute and energy. This geometric approach highlights that the parameter k must be chosen during model building to adjust the decision surface complexity.

Slide 103: • Voronoi diagram: neighboring cells may make the same prediction

A Voronoi diagram partitions the feature space into regions where every point is closer to a specific node than any other, defined by bisector lines between pairs of nodes. While neighboring cells in this diagram may make the same prediction, the decision surface specifically separates regions that yield different predictions. By applying this construction to all training example pairs, the entire space can be partitioned based on proximity to these nodes.

Slide 104: - Small k: each example can have a large effect on prediction → scattered

A small k value creates rough, scattered decision surfaces where individual examples exert a large influence, increasing the risk of overfitting. Conversely, a large k introduces an averaging effect that limits each example's impact, resulting in smoother surfaces but raising the risk of underfitting. Selecting the optimal k requires balancing these opposing risks to avoid losing meaningful patterns while preventing excessive sensitivity to noise.

Slide 105: - Iterate over different ‘k’ values, different distance measures, …

To select optimal k-NN parameters, iterate over different 'k' values and distance measures using the training set as neighbors. Evaluate performance on the validation set to identify the best configuration by tweaking these parameters repeatedly. Once optimized, use the test set solely to evaluate the final model against other approaches, ensuring the training, validation, and test sets serve their distinct roles in parameter selection and comparison.

Slide 106: What should we worry

Slide 106 introduces the topic of potential concerns when utilizing the k-Nearest Neighbors (kNN) algorithm. The slide explicitly poses the question, "What should we worry about when using kNN?" However, no specific issues, definitions, or examples are provided in the accompanying source text or transcript to elaborate on these worries.

Slide 107: Faculteit, departement, dienst …

Slide 107 presents the terms "Faculteit, departement, dienst" and poses the question of whether this sequence constitutes a correct example. No further explanation, definition, or validation is provided in the aligned source material to determine the accuracy of this example.

Slide 108: Faculteit, departement, dienst …

Slide 108 addresses the challenge of calculating distances across multiple scales, specifically questioning which metric applies when variables differ significantly. It highlights that BPM and Energy operate on different domains or scales, noting that BPM values are higher than those for Energy. This discrepancy creates a problem in determining the appropriate distance measure for such heterogeneous data.

Slide 109: - Min-max normalization: Map between 0 and 1 →

Min-max normalization maps data values to a range between 0 and 1. The slide illustrates this by transforming raw BPM and Energy measurements, such as 50 and 1.5, into normalized values like 0 and 0.96. This process scales the original dataset so that the minimum value becomes 0 and the maximum value becomes 1, with intermediate values proportionally adjusted.

Slide 110: - Min-max normalization: Map between 0 and 1 →

Min-max normalization maps attribute values to a range between 0 and 1 by subtracting the variable's minimum and dividing by the difference between its maximum and minimum. For example, with BPM values ranging from 50 to 175, a value of 170 normalizes to 0.96. This technique is preferred when distance matters, whereas standardization alternatively uses the mean and standard deviation to scale data.

Slide 111: Duration

Slide 111 presents two musical examples illustrating duration and classification. The first example has a duration of 210 seconds, is in D Major with a tempo of 137 BPM, and belongs to the Disco-pop class. The second example, labeled as Problem 2, features a duration of 174 seconds, is in A-flat major with a tempo of 186 BPM, and is classified as Punk.

Slide 112: Duration

Slide 112 presents an illustration for "Problem 2" comparing three musical examples by duration, key, tempo (BPM), and genre classification. The first example is a 210-second Disco-pop track in D Major at 137 BPM. The second is a 174-second Punk track in A-flat major at 186 BPM, while the third shares the same duration and key as the second but has an unknown genre at 160 BPM.

Slide 113: Duration

This slide illustrates nearest neighbor classification using song attributes like duration, key, and BPM to categorize the Jackson 5. Although the slide text lists conflicting values, the transcript clarifies that the Jackson 5 share identical duration and key with Green Day's "American Idiot," resulting in a Euclidean distance of 1 compared to ABBA's distance of 2. Consequently, the model classifies the Jackson 5 as punk rock because they are closer to the punk reference point than the disco-pop example.

Slide 114: - Irrelevant features dominate distance

Irrelevant features can dominate distance measures, leading to poor classification performance. To address this, solutions include leveraging domain knowledge to carefully select the feature space, applying weights to prioritize relevant attributes over less important ones like duration, or reducing dimensionality to automatically retain only significant features.

Slide 115: - Weird things happen in high dimensions because our intuitions from low-

The curse of dimensionality arises because intuitions from low-dimensional spaces do not translate to high-dimensional ones, creating significant problems for learners. Adding many features can lead to spurious correlations and mislead kNN algorithms as data density lowers, causing distances to flatten until all examples appear equidistant. This phenomenon occurs when moving from 2D to higher dimensions like 10 or 100, though it does not apply if the distribution of positives has low variance.

Slide 116: - kNN is very slow

k-Nearest Neighbors (kNN) is computationally slow because every query requires computing distances across all training examples, resulting in O(nd) complexity where n is the number of examples and d is the dimension. This approach also necessitates storing all training data in memory. To improve efficiency, solutions include using efficient retrieval structures like KD trees, applying locality sensitive hashing to identify neighbors via hashes, or employing product quantization to compress data and approximate distances.

Slide 117: - Advantages

k-NN offers fast training as a lazy model that defers computation to validation while easily learning complex target functions through varying k values. Its flexibility allows prediction of discrete, real, or probabilistic outputs across diverse data types by simply adjusting the distance measure, unlike specialized neural networks. Additionally, k-NN supports online incremental learning and shares reasoning elements with human cognition.

Slide 118: Data: (X,Y) examples

k-Nearest Neighbors (kNN) is a lazy, instance-based supervised learning model that relaxes exact feature matching by assuming nearby examples share similar outputs. Prediction involves selecting k nearest training instances based on a chosen distance measure and combining their labels, requiring careful choices regarding feature scaling and weighting. While intuitive, kNN faces pitfalls including high computational cost at test time, large memory requirements, and sensitivity to irrelevant features or dimensionality.

Slide 119: Questions?

Slide 119 serves as a concluding prompt inviting questions from the audience. The slide displays the heading "Questions?" alongside the text "Faculteit, departement, dienst," indicating the relevant faculty or departmental context for inquiries. No additional explanatory content or professor commentary is provided for this specific section in the source material.

Key points to remember

The core of the lecture focuses on the mechanics of the kNN algorithm, an instance-based method that classifies a query point by identifying its k closest neighbors in feature space using specific distance metrics such as Euclidean, Manhattan, Hamming, or Jaccard distance. A key exam concept is the necessity of normalizing features to prevent variables with larger numerical ranges from skewing distance calculations. The algorithm aggregates neighbor labels via majority voting or distance-weighted schemes, where closer neighbors exert more influence. The value of k acts as a control for model complexity: small k values create rough decision surfaces prone to overfitting, while large k values produce smoother surfaces that risk underfitting. Geometrically, kNN partitions the feature space into Voronoi regions, and its effectiveness relies heavily on appropriate distance measures and handling the curse of dimensionality, despite advantages like fast training times and flexibility in output types.